

LAPORAN PENGUJIAN PERANGKAT LUNAK

(Software Testing Report)

SVARA — Sistem Pemesanan Tiket Konser

Aplikasi Web berbasis Spring Boot REST API
(Arsitektur Controller – Service – Repository)

Nama Proyek	SVARA (svara2)
Jenis Proyek	Tugas Akademik – Mata Kuliah Pemrograman Berorientasi Objek (PBO)
Anggota Tim	Mutiara Rizky Salsabila (41524010092) Deswita Nindya Putri (41524010082) Keisya Rizkia Kamila (41524010093)
Repositori	project-svara (mutiararzs)
Tanggal Pengujian	23 Juli 2026
Disusun oleh	Claude (Anthropic) – berdasarkan analisis berkas proyek yang diunggah (svara2.zip)

1. Ringkasan Eksekutif

Dokumen ini merangkum hasil pengujian fungsional dan tinjauan keamanan terhadap aplikasi **SVARA**, sebuah sistem pemesanan tiket konser berbasis web yang dibangun dengan Spring Boot (backend REST API) dan HTML/JavaScript murni (frontend statis). Pengujian mencakup seluruh endpoint REST API yang tersedia pada `Svara2Controller`, meliputi autentikasi User dan Admin, manajemen data User (CRUD), serta pencatatan transaksi pembayaran tiket.

Dari **19 skenario pengujian** yang dijalankan, **15 skenario dinyatakan Sesuai (Pass)** terhadap perilaku yang diharapkan, dan **4 skenario dinyatakan Tidak Sesuai (Fail)** – seluruhnya terkait aspek keamanan (validasi input, penyimpanan password, konfigurasi CORS, dan kredensial basis data). Tidak ditemukan cacat fungsional yang menyebabkan sistem gagal total (crash) pada alur utama aplikasi.

***Catatan metodologi:** proyek belum memiliki automated test suite (folder `src/test/java` kosong), dan aplikasi dikonfigurasi untuk terhubung ke basis data PostgreSQL terkelola (Supabase) yang tidak dapat diakses dari lingkungan analisis ini. Oleh karena itu, pengujian pada laporan ini dilakukan melalui **verifikasi kode sumber (white-box / static code tracing)** secara menyeluruh terhadap setiap endpoint, dikombinasikan dengan skenario pengujian fungsional manual yang telah didefinisikan tim pada README proyek. Setiap hasil pada kolom “Hasil Aktual” merupakan hasil penelusuran alur logika kode yang sebenarnya, bukan asumsi. Untuk pengujian eksekusi langsung (live run) di browser/Postman, lihat rekomendasi pada Bagian 6.*

2. Ruang Lingkup dan Metodologi Pengujian

2.1 Ruang Lingkup

- **Full**, backend REST API – seluruh 11 endpoint pada `Svara2Controller`.
- **Full**, logika layer Service (`UserService`, `AdminService`, `PembayaranService`) dan Repository (Spring Data JPA).
- **Full**, mekanisme proteksi akses halaman Admin pada sisi klien (`Admin.html`).
- **Full**, konfigurasi keamanan dasar: CORS, penyimpanan password, dan kredensial basis data.

2.2 Metodologi

Pengujian menggunakan pendekatan **black-box functional testing** (dari sudut pandang kontrak API: input → output yang diharapkan) yang divalidasi dengan **white-box code tracing** (penelusuran langsung kode Java pada Controller, Service, dan Repository) untuk memastikan hasil aktual konsisten dengan implementasi. Pendekatan ini dipilih karena tidak tersedia unit test otomatis pada proyek maupun koneksi ke basis data produksi dalam lingkungan pengujian ini.

2.3 Lingkungan Pengujian

Komponen	Keterangan
Bahasa & Framework	Java 17, Spring Boot 3.4.5 (Web, Data JPA)
Build tool	Maven (Maven Wrapper – mvnw)
Basis data (produksi)	PostgreSQL – hosting Supabase (connection pooler)
Basis data (opsi lokal)	H2 file-based (dinonaktifkan/di-comment di <code>application.properties</code>)
Port aplikasi	9999
Frontend	HTML5 + Tailwind CSS (CDN) + JavaScript vanilla (fetch API)

Deployment	Railway (PaaS) – project-svara-production.up.railway.app
------------	--

3. Kasus Uji Fungsional (Functional Test Cases)

ID	Skenario / Endpoint	Data Uji / Langkah	Hasil Diharapkan	Hasil Aktual (Verifikasi Kode)	Status
TC-01	Registrasi User baru POST /register	Kirim nama, email, password, nomorTelepon, tanggalLahir yang valid & lengkap	User baru tersimpan, response 200 berisi objek User	UserService.simpanUser() langsung memanggil userRepository.save() tanpa aturan tambahan → data tersimpan sesuai kiriman	Sesuai
TC-02	Registrasi dengan data tidak lengkap / format tidak valid	Kirim request dengan field kosong (mis. email kosong) atau format email tidak valid	Sistem menolak dan mengembalikan pesan kesalahan validasi	Tidak ada anotasi @Valid atau pengecekan manual pada Controller maupun Service – data kosong/tidak valid tetap tersimpan ke basis data	Tidak Sesuai
TC-03	Login User – kredensial benar POST /user/login	Email & password sesuai data pada tabel USERS	Response 200, mengembalikan data User yang login	findByEmailAndPassword menemukan baris yang cocok → Controller mengembalikan ResponseEntity.ok(user)	Sesuai
TC-04	Login User – kredensial salah	Email terdaftar tetapi password salah, atau email tidak terdaftar	Response 401 dengan pesan “Email atau password salah”	Query mengembalikan null → Controller mengembalikan status 401 sesuai pesan yang diharapkan	Sesuai
TC-05	Login Admin – kredensial benar POST /admin/login	Email & password sesuai data pada tabel ADMIN	Response 200, mengembalikan data Admin	Pola identik dengan login User: AdminService.loginAdmin() → 200 OK bila ditemukan	Sesuai
TC-06	Login Admin – kredensial salah	Email/password Admin tidak sesuai	Response 401 dengan pesan kesalahan	Hasil null ditangani identik dengan alur login User → 401	Sesuai
TC-07	Ambil seluruh data User GET /users	Tanpa parameter	Response 200, daftar seluruh User	userRepository.findAll() dikembalikan langsung sebagai List<User>	Sesuai
TC-08	Ambil User berdasarkan ID – ditemukan GET /users/{id}	ID valid yang ada di basis data	Response 200, data User sesuai ID	findById().orElse(null) menghasilkan objek → 200 OK	Sesuai
TC-09	Ambil User berdasarkan ID – tidak ditemukan	ID yang tidak ada di basis data	Response 404, pesan “User tidak ditemukan”	Hasil null ditangani Controller → status 404 sesuai harapan	Sesuai
TC-10	Perbarui data User – ditemukan PUT /users/{id}	ID valid + payload nama/email/nomorTelepon/tanggalLahir baru	Response 200, data User terbaru	Field nama, email, nomorTelepon, tanggalLahir diperbarui dan disimpan. Catatan: field password tidak turut diperbarui meskipun dikirim pada payload (tidak di-set ulang pada UserService.updateUser)	Sesuai
TC-11	Perbarui data User – tidak ditemukan	ID yang tidak ada di basis data	Response 404	oldUser == null → method mengembalikan null → Controller 404	Sesuai

ID	Skenario / Endpoint	Data Uji / Langkah	Hasil Diharapkan	Hasil Aktual (Verifikasi Kode)	Status
TC-12	Hapus User – ditemukan DELETE /users/{id}	ID valid	Response 200, pesan "User berhasil dihapus"	User ditemukan → deleteById() dipanggil → mengembalikan true → 200 OK	Sesuai
TC-13	Hapus User – tidak ditemukan	ID yang tidak ada	Response 404	User tidak ditemukan → mengembalikan false → Controller 404 sesuai harapan	Sesuai
TC-14	Simpan transaksi pembayaran POST /pembayaran	namaKonser, kategoriTiket, namaUser, jumlahTiket, metodePembayaran, totalBayar	Transaksi tersimpan dengan tanggalBeli terisi otomatis	PembayaranService mengisi tanggalBeli otomatis memakai SimpleDateFormat("d MMMM yyyy, HH:mm", Locale id-ID) sebelum save() – sesuai harapan	Sesuai
TC-15	Riwayat pembayaran per User GET /pembayaran/riwayat?namaUser=...	Parameter namaUser sesuai transaksi yang tersimpan	Daftar transaksi milik user tsb,urut dari terbaru	findByNamaUserOrderByIdDesc mengembalikan hasil sesuai urutan yang diharapkan. Catatan: pencocokan berbasis string nama, bukan relasi ID/foreign key ke tabel USERS – berpotensi ambigu bila dua user memakai nama sama	Sesuai
TC-16	Proteksi akses halaman Admin (client-side) Admin.html	Akses langsung ke Admin.html tanpa login melalui address bar	Muncul peringatan akses ditolak & redirect ke LoginAdmin.html	Skrip memeriksa sessionStorage.getItem("isAdminLoggedIn") !== "true" lalu redirect – sesuai harapan pada level UI. Catatan: proteksi hanya di sisi klien; endpoint GET /users, PUT, dan DELETE di backend tidak memiliki pengecekan otorisasi apapun dan dapat dipanggil langsung tanpa login	Sesuai

4. Temuan Tinjauan Keamanan (Security Review)

Selain pengujian fungsional, dilakukan tinjauan keamanan dasar terhadap kode sumber. Tiga temuan berikut dikategorikan **Tidak Sesuai** terhadap praktik keamanan yang wajar untuk aplikasi produksi, meskipun tidak mengganggu jalannya fungsi utama aplikasi (fungsional tetap Pass, sebagaimana tercakup pada Bagian 3).

ID	Aspek	Temuan	Status
TC-17	Konfigurasi CORS <code>@CrossOrigin("")</code>	Anotasi ini diterapkan pada seluruh Controller sehingga permintaan dari origin mana pun diizinkan mengakses API – termasuk endpoint sensitif seperti CRUD User dan login Admin.	Tidak Sesuai
TC-18	Penyimpanan Password	Field password pada Entity User dan Admin disimpan sebagai teks biasa (plain text), tanpa hashing (mis. BCrypt). Proses login membandingkan string password secara langsung lewat <code>findByEmailAndPassword</code> .	Tidak Sesuai
TC-19	Manajemen Kredensial Basis Data	Berkas <code>application.properties</code> menyertakan nilai <i>default</i> untuk URL, username, dan password koneksi Supabase secara eksplisit (sebagai fallback dari environment variable). Nilai kredensial aktual tidak dicantumkan ulang pada laporan ini demi keamanan, namun keberadaannya di dalam berkas konfigurasi yang ikut ter-commit ke repository merupakan risiko kebocoran data.	Tidak Sesuai

Ketiga temuan di atas juga telah disadari oleh tim pengembang dan didokumentasikan pada bagian “Batasan Desain” README proyek, sehingga tidak bersifat mengejutkan – namun tetap perlu ditindaklanjuti sebelum aplikasi digunakan di luar konteks akademik.

5. Ringkasan Hasil Pengujian

Kategori	Jumlah Skenario	Sesuai (Pass)	Tidak Sesuai (Fail)
Fungsional (TC-01 s.d. TC-16)	16	15	1
Keamanan (TC-17 s.d. TC-19)	3	0	3
Total	19	15	4

Tingkat kesesuaian fungsional: **15/16 (93,8%)** skenario fungsional berjalan sesuai spesifikasi berdasarkan penelusuran kode. Satu-satunya kegagalan fungsional (TC-02) disebabkan tidak adanya validasi input pada endpoint registrasi. Seluruh 3 aspek keamanan yang ditinjau memerlukan perbaikan sebelum rilis produksi.

6. Kesimpulan dan Rekomendasi

Secara fungsional, aplikasi **SVARA** telah mengimplementasikan alur inti yang dibutuhkan sistem pemesanan tiket konser: registrasi & login User/Admin, manajemen data User (CRUD), pencatatan transaksi pembayaran, dan riwayat pembelian per pengguna – seluruhnya bekerja sesuai harapan pada level logika kode. Rekomendasi berikut disusun untuk meningkatkan kualitas dan keamanan aplikasi:

- 1 Tambahkan validasi input pada backend (anotasi `@Valid` / `@NotBlank` pada DTO, atau pengecekan manual di Service) untuk endpoint registrasi dan pembayaran.
- 2 Terapkan hashing password (mis. BCrypt via `spring-boot-starter-security`) dan hindari perbandingan password secara plain text.

- 3 Batasi @CrossOrigin hanya ke origin frontend resmi, bukan wildcard ("*").
- 4 Pindahkan seluruh kredensial basis data ke environment variable tanpa nilai default di dalam kode, dan rotasi kredensial Supabase yang sempat tercantum di repository.
- 5 Tambahkan otorisasi sisi server (mis. Spring Security / filter token) untuk endpoint admin (/users, PUT, DELETE), agar proteksi tidak hanya bergantung pada sessionStorage di sisi klien.
- 6 Lengkapi proyek dengan automated test (JUnit 5 + Mockito untuk unit test Service, MockMvc@SpringBootTest untuk integration test Controller) agar pengujian dapat dijalankan otomatis pada setiap perubahan kode, tanpa bergantung pada verifikasi manual seperti pada laporan ini.
- 7 Pertimbangkan relasi berbasis ID (foreign key) antara tabel pembayaran dan USERS, alih-alih mencocokkan riwayat transaksi berdasarkan nama (string).

Laporan ini disusun berdasarkan analisis berkas proyek (svara2.zip) yang diunggah, tanpa akses eksekusi langsung ke basis data produksi maupun repositori dependensi Maven Central dari lingkungan pengujian. Untuk validasi akhir sebelum produksi, disarankan menjalankan skenario yang sama secara langsung melalui browser/Postman pada lingkungan lokal atau staging.